

Lecture 5: Dynamic Programming 2

usually the trickiest part

The DP Recipe

1. Define *value* version of what we want to compute
2. Derive a recurrence
3. Size of table: How many dimensions? Range of each dimension?
4. What are the base case(s)?
5. To fill in a cell, which other cells need to be filled already? In which order do I fill the table?
6. Which cell(s) contain the final answer?
7. Running time is often (size of table) $\cdot$ (time to fill each entry)
8. To reconstruct the solution (instead of just its value) follow arrows from final answer to base case

Longest Common Subsequence

Given: 2 strings S_1 of length N and S_2 of length M

Consider an LCS, call it s .

Consider the last symbols of S_1 and S_2

Case 1, use it:

Case 2, lose it:

LCS DP

$$\text{LCS}(S_1[1..N], S_2[1..M]) =$$

$S_1=\text{GAC}; S_2=\text{AGCAT}$						
	ϵ	...A	...G	...C	...A	...T
ϵ						
...G						
...A						
...C						

Knapsack DP

$\text{Knapsack}(\{t_1, \dots, t_i\}, c) =$

remaining weight

items up through		0	1	2	3
	$\emptyset$				
	t₁ : 4\$, 2lb				
	t₂ : 8\$, 1lb				
	t₃ : 6\$, 2lb				
	t₄ : 5\$, 1lb				